

Vetores, strings e passagem por referência

O que uma função recebe quando você entrega um vetor

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Por que o tamanho de um vetor se perde dentro de uma função



Strings: vetores de `char` que terminam em `\0`



Passagem por valor $\times$ por referência (e o papel do `const`)

De onde partimos

Você já sabe que `&` devolve um endereço e `*` devolve o conteúdo. Hoje esses dois operadores explicam o comportamento de vetores, strings e parâmetros de função.

Vetores em funções

O problema

Uma função que processe um vetor precisa saber *onde* ele começa e *quantos* elementos tem.

Mas `sizeof` — que funciona tão bem em `main` — devolve a resposta errada dentro da função.

A pista

Um vetor não é copiado quando você o passa para uma função. O que a função recebe é bem menor do que você imagina.

Descobrir o que ela recebe explica, de uma vez, vetores, strings e passagem por referência.

```
1  #include <stdio.h>
2  #define MAX_CONTATOS 4
3  #define MAX_NOME      20
4
5  void medir(int idades[], int n) {
6      printf("A: sizeof dentro da funcao = %zu | n = %d\n", sizeof(idades), n);
7  }
8
9  void envelhecer(int idades[], int n) {
10     for (int i = 0; i < n; i++)
11         idades[i] = idades[i] + 1;
12 }
```

```
1  int meu_tamanho(const char *s) {
2      int n = 0;
3      while (s[n] != '\0')
4          n++;
5      return n;
6  }
7
8  void maiusculas(char *s) {
9      for (int i = 0; s[i] != '\0'; i++)
10         if (s[i] >= 'a' && s[i] <= 'z')
11             s[i] = s[i] - 'a' + 'A';
12 }
```

```
1 void estatisticas(int idades[], int n, int *soma, int *maior) {
2     *soma = 0;
3     *maior = idades[0];
4     for (int i = 0; i < n; i++) {
5         *soma = *soma + idades[i];
6         if (idades[i] > *maior)
7             *maior = idades[i];
8     }
9 }
```

```
1  int main(void) {
2      char nomes[MAX_CONTATOS][MAX_NOME] = {"ana", "bruno", "carla", "daniel"};
3      int idades[MAX_CONTATOS] = {21, 19, 23, 20};
4      int soma, maior;
5      printf("A: sizeof em main = %zu\n", sizeof(idades));
6      medir(idades, MAX_CONTATOS);
7      envelhecer(idades, MAX_CONTATOS);
8      printf("B: idades = %d %d %d %d\n", idades[0], idades[1], idades[2], idades[3]);
9      printf("C: sizeof(nomes[1]) = %zu | meu_tamanho = %d\n",
10             sizeof(nomes[1]), meu_tamanho(nomes[1]));
11      maiusculas(nomes[1]);
12      printf("D: nomes[1] = %s\n", nomes[1]);
13      estatisticas(idades, MAX_CONTATOS, &soma, &maior);
14      printf("E: soma = %d | maior = %d\n", soma, maior);
15      return 0;
16 }
```

Antes de compilar: escreva sua previsão no papel

1. Na linha **A**, os dois `sizeof` dão o mesmo número?
2. Na linha **B**, as idades mudaram? `envelhecer` recebeu idades sem nenhum `&`.
3. Na linha **C**, `sizeof(nomes[1])` e `meu_tamanho(nomes[1])` dão o mesmo número?
4. Na linha **D**, `nomes[1]` virou maiúsculo de verdade?
5. Na linha **E**, por que `soma` e `maior` precisaram de `&` e `nomes[1]` não precisou?

Anote as cinco respostas. Elas são o roteiro do resto da aula.

```
$ gcc -Wall agenda.c -o agenda && ./agenda
```

```
A: sizeof em main = 16  
A: sizeof dentro da funcao = 8 | n = 4  
B: idades = 22 20 24 21  
C: sizeof(nomes[1]) = 20 | meu_tamanho = 5  
D: nomes[1] = BRUNO  
E: soma = 87 | maior = 24
```

Marque o que surpreendeu

16 contra 8; 20 contra 5; e um vetor alterado por uma função sem & nenhum.

Por que `sizeof` deu 16 fora e 8 dentro?



Os números

- Em `main`: $4 \times 4 = 16$ bytes — o vetor inteiro.
- Em `medir`: 8 bytes.

8 bytes é o tamanho de quê?

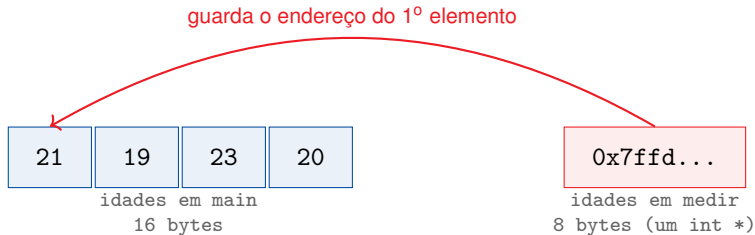
Do *endereço* — o mesmo tamanho de qualquer ponteiro nesta máquina.

O próprio compilador avisou

```
warning: sizeof on array function  
parameter will return size of 'int *'  
instead of 'int[]'
```

Leia os avisos do `-Wall`: aqui ele descreve exatamente o que aconteceu.

A resposta: o vetor “decai” em ponteiro



Regra

Ao ser passado para uma função, o nome do vetor vira o **endereço do primeiro elemento**. A função recebe um ponteiro — nunca uma cópia do vetor.

```
void envelhecer(int idades[], int n)    /* o que escrevemos */  
void envelhecer(int idades[4], int n)  /* o 4 e' ignorado pelo compilador */  
void envelhecer(int *idades, int n)    /* o que o compilador entende */
```

Consequências práticas

- As três formas são **idênticas** para o compilador.
- Por isso o parâmetro `n` não é opcional: sem ele a função não tem como descobrir o tamanho.
- `idades[i]` dentro da função é apenas outra escrita para `*(idades + i)`.

Não há mistério

A função recebeu o endereço do vetor.
Escrever em `idades[i]` é escrever *naquele* endereço — ou seja, na memória de `main`.

A assimetria de C

- `int x` — a função recebe uma cópia; alterações se perdem.
- `int v[]` — a função recebe um endereço; alterações permanecem.

Cuidado

Toda função que recebe um vetor pode modificá-lo, mesmo sem você querer.
Mais adiante veremos como o `const` declara essa intenção.

```
void errado(int v[]) {  
    int n = sizeof(v) / sizeof(v[0]);    /* 8 / 4 = 2. Sempre errado! */  
    for (int i = 0; i < n; i++) ...  
}  
  
void certo(int v[], int n) {              /* quem chama informa o tamanho */  
    for (int i = 0; i < n; i++) ...  
}
```

Onde a conta `sizeof(v)/sizeof(v[0])` funciona

Só no escopo em que o vetor foi **declarado** — tipicamente em `main`. Calcule lá e passe o resultado adiante.

O que aprendemos

- O nome de um vetor, ao ser passado, vira o endereço do primeiro elemento (decaimento).
- `int v[]`, `int v[4]` e `int *v` são o mesmo parâmetro.
- O tamanho não viaja junto: passe-o como um parâmetro extra.
- Quem recebe um vetor pode alterar o original — e isso é útil, não um defeito.

Strings

Por que `sizeof` deu 20 e `meu_tamanho` deu 5?



Os dois números

- `sizeof(nomes[1]) = 20`: a **capacidade** reservada na declaração.
- `meu_tamanho(nomes[1]) = 5`: o **conteúdo** atual, "bruno".

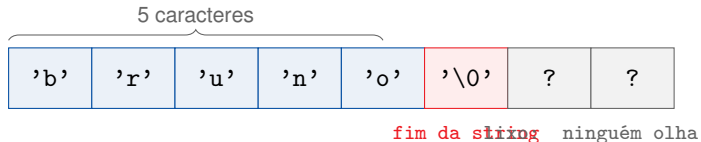
A pergunta

Se `meu_tamanho` só recebeu um endereço, como ela soube onde a palavra termina?

A resposta é uma convenção — e é a única coisa que faz strings funcionarem em C.

Definição

Uma **string** é um vetor de `char` cujo fim é marcado pelo caractere `'\0'` (o byte de valor zero, chamado *terminador nulo*).



Intuição

O vetor tem 20 posições; a string ocupa 6 delas (5 letras + o terminador).

```
int meu_tamanho(const char *s) {  
    int n = 0;  
    while (s[n] != '\0')    /* anda ate' encontrar o terminador */  
        n++;  
    return n;               /* nao conta o '\0' */  
}
```

Por que isso basta

O ponteiro diz onde a string *começa*; o `'\0'` diz onde ela *acaba* — mesma estratégia de `strlen` e de `printf("%s", s)`. Sem o terminador, a função lê memória que não é dela até achar um zero por acidente.

nomes[0]	a	n	a	\0	?	?	?
nomes[1]	b	r	u	n	o	\0	?
nomes[2]	c	a	r	l	a	\0	?
nomes[3]	d	a	n	i	e	l	\0

... até 20 colunas por linha

Lendo a declaração

`char nomes[4][20]` = 4 strings de até 19 caracteres + terminador. `nomes[1]` é a linha inteira — e, ao ser passada, decai em `char *`.

```
char a[20] = "ana";    /* copia as letras para 20 bytes SEUS: pode alterar */
char *b      = "ana";  /* aponta para um literal na memoria so' de leitura */

a[0] = 'A';            /* ok -> "Ana" */
b[0] = 'A';            /* ERRO em execucao: segmentation fault */
```

Regra prática

Vai modificar? Declare como vetor.

Só vai ler?

`const char *b = "ana";` — aí o erro aparece na *compilação*, não na execução.

Função	O que faz	Cuidado
strlen(s)	conta até o '\0'	não conta o terminador
strcpy(d, o)	copia o para d	d precisa ter espaço
strcmp(a, b)	compara	devolve 0 se iguais
strcat(d, o)	concatena o no fim de d	espaço para os dois

```
if (strcmp(nomes[i], "bruno") == 0)    /* == 0 significa IGUAIS */  
    printf("achei\n");
```

Nunca use == entre strings

`nomes[i] == "bruno"` compara **endereços**, não conteúdo — e dá falso quase sempre.

O que aprendemos

- String é vetor de `char` terminado em `'\0'`; a marca é o que define o fim.
- `sizeof` dá a capacidade declarada; `strlen` dá o conteúdo atual.
- Passar uma string é passar um `char *` — portanto quem recebe pode alterar o original.
- Literais (`char *s = "..."`) não podem ser modificados; comparação se faz com `strcmp`.

Passagem por valor e por referência

A chamada com &

`estatisticas(idades, 4, &soma, &maior);`
soma e maior são int — variáveis comuns.
Sem &, a função receberia cópias.

A chamada sem &

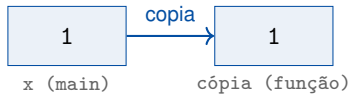
`maiusculas(nomes[1]);`
nomes[1] é um vetor — **já** decai em endereço. Um & a mais seria endereço de endereço.

A regra por trás das duas

C só tem **um** mecanismo de passagem. Descobrir qual é dissolve a aparente exceção.

A resposta: C passa tudo por valor

Passagem por valor



alterar a cópia não muda o original

Passagem do endereço



*soma = 87 escreve na memória de main

A regra única

A função sempre recebe uma **cópia do argumento**. Quando o argumento é um endereço, a cópia aponta para o mesmo lugar — e é assim que C simula a “passagem por referência”.

```
void estatisticas(int v[], int n, int *soma, int *maior);  
/*          entradas          <-- saídas: escrevemos via * */  
  
int soma, maior;  
estatisticas(idades, 4, &soma, &maior); /* & converte em endereço */  
printf("E: soma = %d | maior = %d\n", soma, maior);
```

Convenção que você vai reencontrar sempre

`return` devolve **um** valor. Para devolver vários — ou para separar resultado de código de erro — C usa ponteiros como *parâmetros de saída*.

```
int  meu_tamanho(const char *s);    /* prometo nao alterar a string */
void maiusculas(char *s);           /* aviso que vou alterar          */

/* dentro de meu_tamanho: */
s[0] = 'X';    /* erro de COMPILACAO: assignment of read-only location */
```

Por que usar

- A assinatura documenta a intenção: quem lê a declaração já sabe se seu dado corre risco.
- O compilador vira seu revisor: quebra do contrato não chega a virar bug em execução.

Quero que a função...	Parâmetro	Exemplo
só leia um número	<code>int x</code>	<code>soma(int a, int b)</code>
altere um número	<code>int *x</code>	<code>estatisticas(..., int *soma)</code>
só leia um vetor/string	<code>const T *v, int n</code>	<code>meu_tamanho(const char *s)</code>
altere um vetor/string	<code>T *v, int n</code>	<code>envelhecer(int v[], int n)</code>

Regra de bolso

Vetores e strings você passa “de graça” (só o endereço viaja). Tipos simples exigem `&` explícito quando devem ser alterados.

```
char *saudacao_errada(void) {  
    char msg[20] = "ola";  
    return msg;           /* msg morre no return: endereco invalido */  
}  
  
void saudacao_certa(char *destino, int cap) {  
    if (cap > 3) strcpy(destino, "ola"); /* quem chama fornece o espaco */  
}
```

Padrão de C

Quem chama é dono da memória; a função apenas preenche. Por isso tantas funções recebem um destino e a capacidade dele.

O que aprendemos

- C só passa por valor; “por referência” é passar uma cópia do endereço.
- Tipos simples que devem mudar exigem & na chamada e * no uso.
- Vetores e strings já chegam como endereço — alterá-los é o comportamento padrão.
- `const` documenta e faz o compilador cobrar a promessa de não alterar.

Mãos à obra

Tarefa 1

Escreva `void minusculas(char *s)`, a inversa de maiúsculas, e aplique-a a `nomes[3]`. Imprima o resultado.

Tarefa 2

Acrescente `void media(const int v[], int n, float *m)` e imprima a média das idades com uma casa decimal. Por que `const` cabe aqui e não em envelhecer?

Tarefa 3

Escreva `int busca(char nomes[][MAX_NOME], int n, const char *alvo)` devolvendo o índice do contato ou `-1`. Use `strcmp`.

Agenda ordenada

Escreva `void ordena_nomes(char nomes[][MAX_NOME], int idades[], int n)` que ordene os contatos em ordem alfabética **mantendo cada idade junto do seu nome**.

Requisitos

- Compare nomes com `strcmp` e troque strings com `strcpy` e um vetor auxiliar.
- Toda troca de nome deve vir acompanhada da troca da idade.
- A função não devolve nada: o efeito é nos vetores de `main`.

As cinco respostas

1. 16×8 : o vetor decai em ponteiro.
2. Sim: a função escreveu na memória de `main`.
3. 20×5 : capacidade $\times$ conteúdo.
4. Sim: `char *` aponta para o original.
5. Vetores já são endereços; `int` precisa de `&`.

Próxima aula

Se o vetor precisa existir com tamanho decidido em execução, quem reserva essa memória? Entra `malloc` — e o ponteiro deixa de apontar para algo que já existe.

Para praticar

Tutorial da aula: tarefas 1–3 e o desafio, com testes automáticos.



André Backes.

Linguagem C: completa e descomplicada.

Elsevier, Rio de Janeiro, 2013.



Randal E. Bryant and David R. O'Hallaron.

Computer Systems: A Programmer's Perspective.

Pearson, Boston, 3 edition, 2015.

Seção 3.9: alinhamento de dados e layout de structs.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Algoritmos: teoria e prática.

Elsevier, Rio de Janeiro, 3 edition, 2012.



Paul Deitel and Harvey Deitel.

C: como programar.

Pearson Prentice Hall, São Paulo, 6 edition, 2011.



Ulrich Drepper.

What Every Programmer Should Know About Memory.

Red Hat, Inc., 2007.

Seção 6.2: efeito da ordem de percurso de matrizes sobre a cache.



Ulrich Drepper.

How to write shared libraries.

Documento técnico, 2011.

Versão 4.1.2.



Paulo Feofiloff.

Algoritmos em linguagem C.

Elsevier, Rio de Janeiro, 2009.



ISO/IEC.

Iso/iec 9899:2018 — information technology: Programming languages: C.

Technical report, International Organization for Standardization, 2018.

Seção 6.7.2.1: structure and union specifiers.



Brian W. Kernighan and Dennis M. Ritchie.
C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.



K. N. King.
C Programming: A Modern Approach.
W. W. Norton, New York, 2 edition, 2008.
Capítulo 16: estruturas, uniões e enumerações.



Robert Mecklenburg.
Managing Projects with GNU Make.
O'Reilly Media, 3 edition, 2004.



Herbert Schildt.
C completo e total.
Makron Books, São Paulo, 3 edition, 1997.

-  Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.
Ferramenta usada no tutorial para detectar vazamentos e uso após free.
-  Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.
-  Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles.
Dynamic storage allocation: A survey and critical review.
In *International Workshop on Memory Management (IWMM)*, pages 1–116. Springer, 1995.
Panorama de estratégias de alocação dinâmica e fragmentação.
-  Nivio Ziviani.
Projeto de algoritmos: com implementações em Pascal e C.
Cengage Learning, São Paulo, 3 edition, 2011.